

NOTICE TECHNIQUE

Phase 1 : Première approche (Échec et complexe)

Destiné à : Jean-Christophe Dubacq, Micaela Mayero et Haïfa Zargayouna

La méthode de construction

Pour cette phase, nous avons choisi une approche simplifiée et géométrique basée sur notre première compréhension des principes de Voronoi.

Le point le plus important qu'on a noté est que chaque frontière entre les points c'était la médiatrice entre ces deux points.

1^{ère} étape :

On utilise la fonction *lire_points* pour ouvrir le fichier *.txt*. Il parcourt chaque ligne et utilise *split*(« , ») pour extraire les coordonnées. On a aussi utilisé *float()* pour accepter les coordonnées précises.

```
def lire_points(fichier):  
    points = []  
    f = open(fichier, 'r')  
    for ligne in f:  
        coordonnees = ligne.split(",")  
        x = float(coordonnees[0])  
        y = float(coordonnees[1])  
        points.append((x,y))  
    f.close()  
    return points
```

2^{ème} étape :

Pour chaque paire de points on calcule :

1. Le point du milieu *M*

```
xm = (point_actuel[0] + point_suivant[0]) / 2  
ym = (point_actuel[1] + point_suivant[1]) / 2
```

2. Le vecteur entre les deux points

```
vx = point_suivant[0] - point_actuel[0]  
vy = point_suivant[1] - point_actuel[1]
```

3. Et le vecteur perpendiculaire

```
vx_perpendiculaire = -vy  
vy_perpendiculaire = vx
```

3^{ème} étape :

C'est la partie la plus complexe du code, l'idée est de trouver tous les points où deux médiatrices se croisent, les intersections. Pour ça on transforme chaque droite en équation cartésienne $Ax + By = C$

Où A est $-vy$ et B est vx

```
A1 = -vy1  
B1 = vx1
```

Et $C = A * xm + B * ym$

```
C1 = A1 * xm1 + B1 * ym1
```

Ensuite on utilise Cramer donc on calcule le déterminant $A1 * B2 - A2 * B1$, si $D = 0$, alors les droites sont parallèles et on ignore, sinon on trouve x et y avec les formules de Cramer.

```
determinant = A1 * B2 - A2 * B1  
  
# "On peut trouver une intersection seulement si [...] !=  
# 0 (sinon les droites sont parallèles)"  
  
if determinant == 0:  
    return None  
  
# Cramer pour trouver X et Y  
x = (C1 * B2 - C2 * B1) / determinant  
y = (A1 * C2 - A2 * C1) / determinant
```

Architecture et choix techniques

Le code est divisé en plusieurs fonctions pour mieux structurer le code, et séparer le calcul de l'affichage :

- *lire_points* : S'occupe de récupérer les coordonnées dans fichier.txt
- *préparer_donnees_voronoi* : Cette fonction vas s'occuper de la géométrie. Elle génère la liste des milieux et des vecteurs mais elle ne dessine pas.
- *tracer_diagramme_voronoi* : Reçoit les données calculées et vas s'occuper de la génération du fichier SVG avec *drawsvg*.
- *calculer_intersections* : Qui vas être purement mathématique (Cramer)

On choisit aussi de calculer toutes les intersections possibles et médiatrices. Parce que c'est une méthode qui paraissait plus logique vu le temps qu'on avait, mais le problème est que ça fait apparaitre trop de lignes bleus et points verts qui ne font pas partie du Voronoi.

Analyse des limites et complexité

Complexité :

On compare chaque point de départ (n) à tout les autres pour créer les médiatrices, donc ici on a une complexité de $O(n^2)$ et on a environ n^2 droites .

Après, pour trouver les intersections, on vas comparer toutes ces médiatrices entre elles. Et comme on boucle déjà n^2 droites, la complexité devient $O(n^4)$.

Par exemple pour 8 points, on montait à plus de 1500 intersections calculées. Donc c'est un programme qui vas être lent si on met énormément de points.

Au cours du développement, on a tenté de coder une fonction *filtrer_intersections* (Commenté dans le code). Le but c'était de garder que les points verts qui sont vraiment des intersections de Voronoi (ceux qui sont à égale distance des trois points noirs les plus proches)

Mais on a abandonné cette piste pour plusieurs raisons :

- Complexité : En codant et en réfléchissant à la suite, c'était devenu beaucoup plus compliqué que prévu notamment pour reconstruire les zones. Avec le temps disponible et le peu de ressources qu'on a trouvé, c'était devenu difficile d'avancer.

- On s'est aussi dit que si une zone de Voronoi était grande, elle n'allait peut-être pas être bien construite à cause du cadre de 500x500, et donc plusieurs problèmes se rajoutaient au problème existant.
- En termes de performance, la double boucle imbriquée était aussi un handicap.